



Semantic segmentation of LiDAR point clouds: Optimising
KPCnv for smart city applications

a dissertation submitted for the degree of Electrical and Electronic Engineering

Daniel Hawkins

210184925

1st May 2026

Word count: 5958

Self-reflection and feedback

Complete this section so that your examiners can provide tailored feedback to your work (up to 50 words per answer). This is a reflective narrative for feedback purposes only, your marks will not be informed by information on this page.

- 1) I believe a strong part of my submission is: The iterative optimisation of KPConv parameters. Finding creative solutions by implementing a dynamic shift in the use of losses was interesting and turned out successful. MATLAB plots and CloudCompare maps meant it could be visualised.

- 2) An area I struggled with is: Implementing dynamic class weightings and Lovász-Softmax loss required extensive debugging. It was hard to understand why dynamic class weightings gave a performance trade-off instead of a general improvement.

- 3) I would particularly like feedback/help with: If the iterative approach was good enough for demonstrating parameter changes or if isolating one variable at a time would've been better. Is 50 epochs a good trade-off between time efficiency and finding general deep learning model trends.

Engineering skills portfolio indexing

Your dissertation needs to demonstrate you have achieved certain learning outcomes as a result of your work which you must reflect on. Please use this checklist to index the section number(s) which demonstrate the appropriate skill. This page **must be filled out** and included in the final report to pass the project.

Engineering skill	Page(s) where demonstrated
Environmental and societal impact Where in your report have you demonstrated that you have considered the wider impact on society to the broader area that you have been working in? For example, if your project linked to reduction of greenhouse gasses or personal mobility, how does your work fit into these broader societal goals?	5-7
Ethical considerations All projects contain ethical considerations, although some are more obvious than others. Have you researched an potential negative impacts of your project? Are there ethical questions in literature or journalism about the area you are working in? What is the general consensus of the engineering community of these issues?	8
Risk management and mitigation For example, have you planned for unexpected outcomes or circumstances? How did you manage changes to your project? Did you decide to change your project and if so how did it change the risk profile of the work? This is usually achieved through regular review, reflection and modification of your project plan, but you must demonstrate this.	20-27
Intellectual property security and data management Whilst some sectors have a more obvious security aspect, all projects require a degree of data protection and management. How have you ensured that the data or intellectual property (IP) which you have generated within your project is secure from external actors or accidental loss? Have you used backups or password protected file stores? Where have you demonstrated an awareness of how IP is captured. How was information management included in part of your project planning?	20
Equality, diversity and inclusion Linking to your considerations of societal impact, where have you considered the inclusivity of your work? Have you demonstrated that you have considered how your work may impact different groups and made efforts to minimise this? Is your work accessible to a wide range of users and could others from a different background to yourself repeat your results based on what you have written in your report?	8
Quality of information Where in your report have you discussed the <i>quality</i> of the information you have researched or the data you have presented? Have you reflected on how this could be improved?	20
Continual professional development and self-reflection Where in your report have you reflected on your own performance throughout the project? Have you identified personal strengths and achievements as well as weaknesses? Have you made plans for exploiting the strengths and improving the weaknesses?	23

Table of Contents

Abstract	5
Aims and objectives	5
Literature review	5
Motivation	5
Considerations.....	7
Principles of LiDAR Technology	8
Raw Point Clouds	8
Early solutions in point cloud deep learning.....	9
Voxel-based methods	10
Multi-view-based approach.....	11
Direct point cloud learning.....	11
PointNet	11
Point-transformers.....	13
KPConv	14
Theoretical Background	15
The KPConv operation	15
Intersection over union and the Lovász-Softmax loss	17
Network layers	18
Methodology and Results	20
Conclusion	29
References	30

Abstract

This project investigates the optimisation of a KPConv deep learning model for semantic segmentation of LiDAR point cloud data applied to a Toronto3D urban dataset. Raw point clouds present challenges such as their non uniform, unstructured and permutation invariant nature, making direct feature extraction inherently difficult. Through iterative modification of training parameters, loss functions and subsampling strategies, the mean intersection over union (mIoU) improved from 0.57 to 0.76. Three optimised models were identified demonstrating performance trade-offs for different priorities and using CloudCompare, a cloud map visualisation can be used for real-world smart city applications.

Aims and objectives

This project optimises a KPConv deep learning model for semantic segmentation using LiDAR data. The Toronto3D dataset represents a small-scale urban constrained to a neighbourhood as to remain with computational limits whilst also capturing class diversity in an urban environment for meaningful evaluation [1].

Objectives

- Preprocess the Toronto3D dataset for KPConv training
- Iteratively modify the training parameters and loss functions to improve mIoU metrics
- Generate a 3D mapping visualisation for observation
- Identify the best performing model configuration through systematic comparisons through evaluation.

Literature review

Motivation

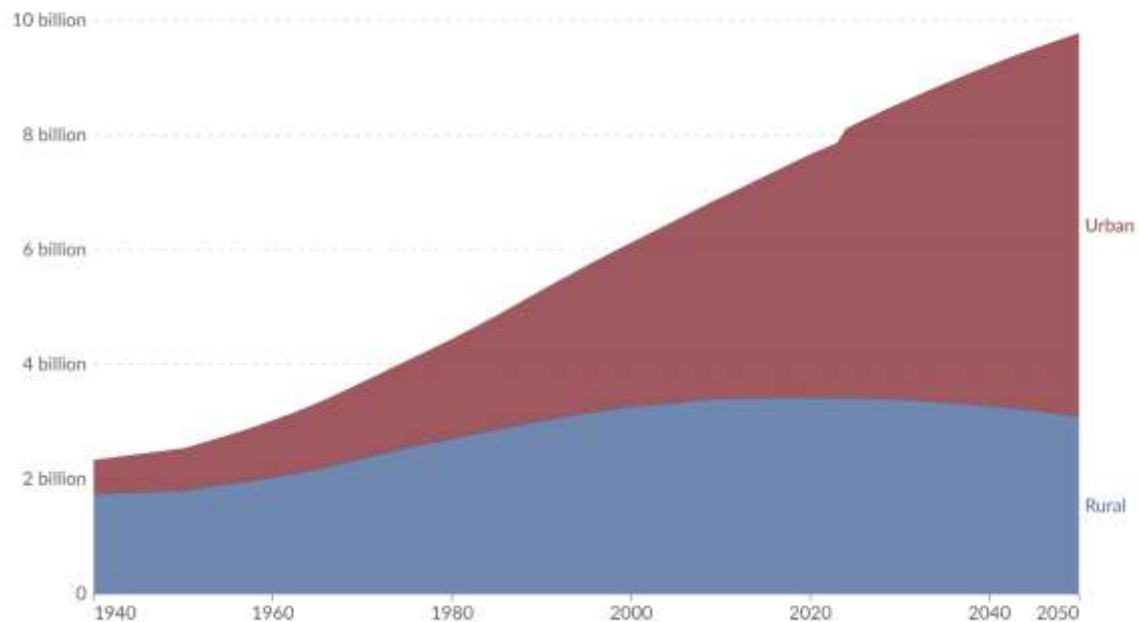
The lack of automated semantic understanding of LiDAR point cloud data limits the development of scalable digital urban models for intelligent city analysis [2]. Raw point cloud data is unstructured, dense and non-uniform which requires deep learning to interpret this information efficiently. Creating a digital model of a city will help improve urban planning for issues such as overcrowding, public transit expansion, environmental degradation and social equity [3].

According to UN projections, by 2050 there will be an estimate of 9.77 billion people worldwide, with 68.4% of the population in an urban environment, and a trend of an increasing rate of population growth [4]. This estimate was based off the UN's medium fertility scenario however fertility rates and fertility projection accuracy can be varied for each country depending on its quality of data and over/underestimates. Analysing past trends therefore must be taken with caution.

Urban and rural population projected to 2050, World, 1940 to 2050

Our World
in Data

Total urban and rural population, given as estimates to 2023, and UN projections to 2050. Projections are based on the UN World Urbanization Prospects and its median fertility scenario.



Data source: United Nations, Department of Economic and Social Affairs, Population Division (2018); HYDE (2023)
OurWorldinData.org/urbanization | CC BY

Figure 1: Projected population of urban and rural populations to 2050

Different countries define an urban environment differently, so when using minimum population thresholds as a definition; problems will still arise. For example, Sweden defines an urban environment with 200 inhabitants however Japan defines an urban environment with a minimum of 50,000 inhabitants [4].

The urban population growth has a much greater weighting than the rural population when it comes to the population increase projection. Figure 2 demonstrates how the global population has shifted from mainly rural to urban areas in present time. More complex and intelligent methods of city planning and urban sprawl will be needed to accommodate a further 1.83 billion people by 2050.

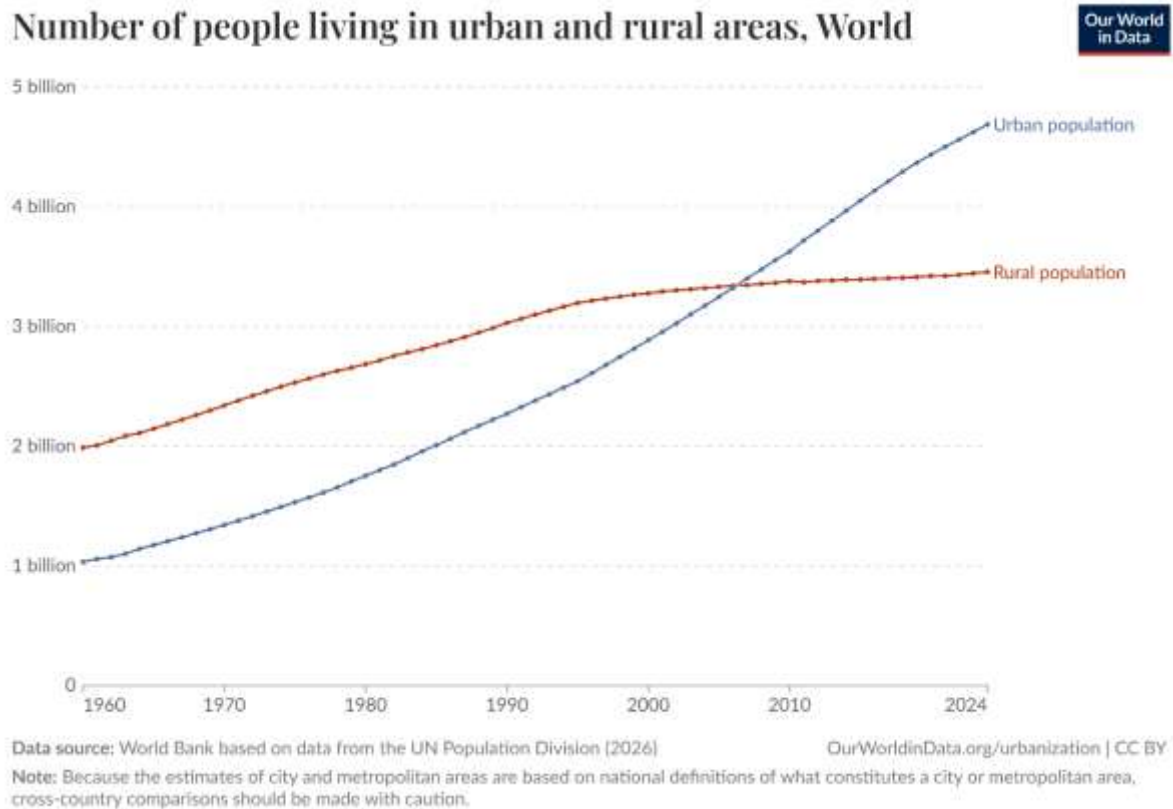


Figure 2: Urban vs Rural population figures

The United Nations Sustainable Development Goals (SDG) are targets towards global development. This is to keep countries accountable for their own national growth and one of the goals are to make cities inclusive, safe, resilient and sustainable [5]. There are sub-goals including reduce adverse effects of natural disasters, inclusive and sustainable urbanisation, reduce environmental impact of cities etc.

There is no universal definition for a smart city however there are general agreed outcomes of one which is smart city planning, enhancing quality of life for residents and sustainability with a clear focus on using digital technology and data analytics [6]. This includes the management and maintenance of infrastructure such as buildings, bridges and roads. Using digital methods to monitor structural health and find diagnosing problems quicker than traditional methods, saving time and money. In terms of risk management for natural disasters, the goal is to adopt and implement a holistic disaster risk management for all levels. There needs to be a system in place to monitor all these sub-goals for SDG 11 [7].

Considerations

There are privacy concerns with LiDAR technology [8] as it may be eventually able to even capture details such as car licence plates and other sensitive features. This could lead to malicious tracking and constant surveillance, so it is important there is transparency to the public about dataset collection and model usage.

Although digital urban models can help sustainability goals, training these models comes at an environmental cost. Multiple training sessions are needed and if scaling to a whole city, data centres may be needed for this computation which are massive energy sinks and use lots of water for cooling [9].

A lot of the datasets available are based on richer countries and do not include countries that may have completely different infrastructure layouts. These other countries are underrepresented and can lead to a neglect of certain needs for different neighbourhoods as models trained on a certain dataset tend to perform worse on other sets [10].

Principles of LiDAR Technology

From the 1930s to early 1960s, aerial photographs were widely used for interpreting the surface of the earth and its geological features. Aerial photographs had improved the mapping space by providing topographic information, spectral information and general interpretation inaccessible to human vision [11]. The main issue was that the analogue format could not be calibrated and weather conditions were vital to the accuracy of these photos.

Remote sensing is a method that uses electromagnetic energy to record and measure characteristics of a target i.e. topology of the earth. Remote sensing offers a greater qualitative analysis such as providing theoretical modelling, spectral measurement and digital image analysis.

LiDAR (Light detection and ranging) is a remote sensing method using laser light to measure distances and create 3-D information for shape and surface characteristics. LiDAR technology transmits a light wave and receives the reflected light wave back; it measures distance by taking the time difference of the transmission to reflection. Airborne LiDAR system consists of a laser range finder and an orientation system that measures the location and orientation of the system. This allows environments to be measured in 3D spaces without needing geographic referencing that aerial photography required [11].

Raw Point Clouds

There are many ways to represent 3D data with meshes, depth images, voxels and point clouds, with point clouds being the most common [12]. This is a collection of points represented in x, y, z coordinates in 3D space. Point clouds are the format that LiDAR scanners capture, they are unique to the other representation methods as they have a more densely sampled representation.

The aim for deep learning is to make sense of the raw point cloud; it must be implemented to sort through the large-scale unordered dataset to infer spatial adjacency relationships between points. There are two main tasks for deep learning: classification and segmentation. Classification is taking a point cloud and assigning a label (class) to it depending on its learned features during model training (e.g. Labelling a point cloud as a car, building, bench etc.). Segmentation is when taking a point cloud

and assigning labels to each different point within itself. Within segmentation there are also two main subcategories: part segmentation and semantic segmentation. Semantic segmentation refers to labelling and identifying multiple objects in a point cloud and part segmentation refers to identifying different parts of the same object. Segmentation is generally harder than classification as you must separate the foreground from the background [13]. There are many different architectures using different methods to improve the capture of local geometry, improve memory bottlenecks and have a robustness to noise [14]. The goal of these architectures is that once trained and extracted local features from training data, it should be able to predict labels for new data that it hasn't seen before. This is the most important use case for these models as the better it is at correctly predicting labels for new data, the better its generalisation capability, which is the ultimate measure of a successful model [15].

Point clouds are unordered and extremely dense making interpreting them manually extremely challenging. Commonly represented in x, y, z coordinates, they are subject to incompleteness as it is impossible for the sensors to capture every angle in a wide urban environment, reflective surface properties and noise [16]. The points are unordered in a file and by changing the order of the file should not change the nature of the scene so must be permutationally invariant [17].

Point clouds are irregular as points are not evenly distributed across different parts of a scene so there can be a fluctuation in point density, which cause the neighbours of a point to be inconsistent across a scene making local geometric extraction unreliable. This can be managed with sub-sampling techniques but cannot be fully removed [18].

Point clouds are unstructured so the distance between neighbouring points will vary causing inconsistent local context. The same geometric structure produces different point cloud representations depending on its distance from the LiDAR sensor. Applying direct kernel convolution against unstructured data will provide inconsistent, unreliable shape information [19], unlike photogrammetry as each pixel has a fixed image-space relationship on a regular grid.

Convolutional neural networks cannot process this raw format because convolution depends on data that is ordered, regular and on a structured grid.

Early solutions in point cloud deep learning

Initially, the solution to these challenges was structured grid-based learning. The convolution operation is used for feature learning so the raw point cloud data must be converted into a structured form [20].

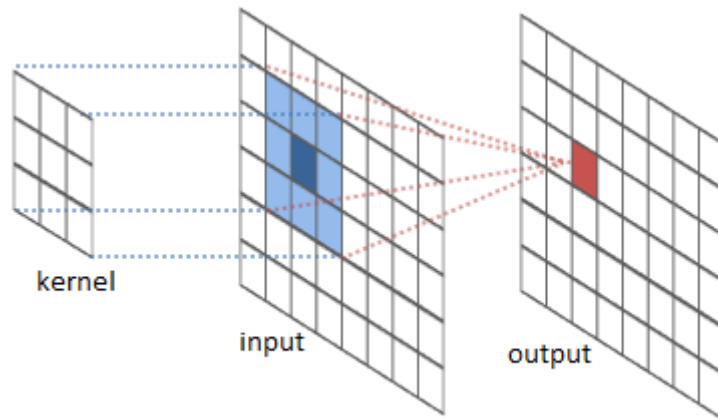


Figure 3: 2D convolution on a regular grid

Voxel-based methods

Voxel based methods convert point cloud structures into a 3D voxel structure and convolves with 3D kernels.

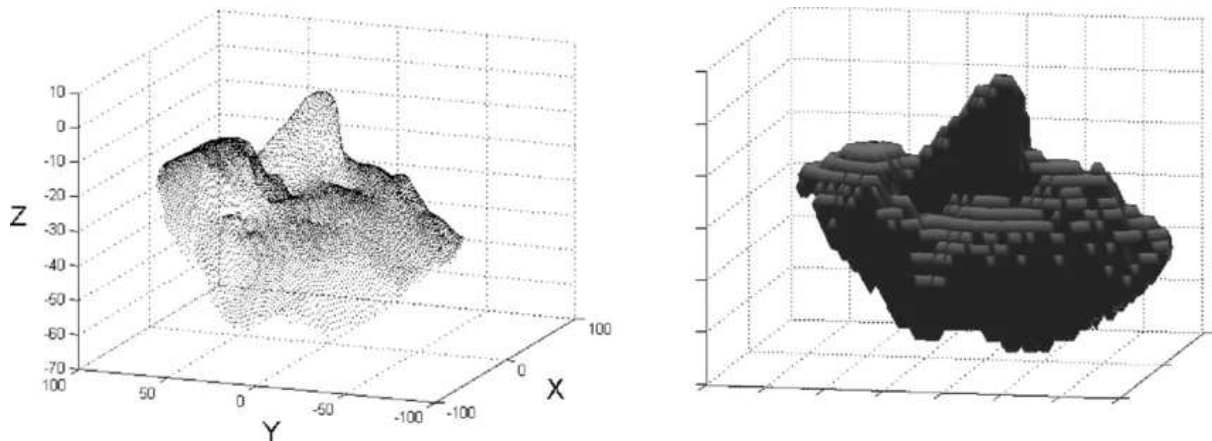


Figure 4: Point cloud to binary voxels

In this method there are two steps, offline processing and online learning. The offline step converts the point cloud to fixed-size voxels as in figure 4. Binary voxels are normally used as they are the simplest type of voxel and represent either being occupied or not [21]. In the learning stage, 3D convolutional neural networks (CNNs) would be applied for training and then used for predictive labelling.

Memory is committed to the full 3D grid, therefore lots of memory power is needed and a lot of it is wasted as most binary voxels are unoccupied and the grids are predominantly empty. This bottleneck has led to other methods being developed to process raw point cloud data.

Although voxel methods first made 3D CNNs possible, scaling them to an urban landscape would be extremely unpractical especially when most of the voxels would be empty leading to huge inefficiency.

Multi-view-based approach

These methods take 2D CNNs and apply them in 3D spaces. Photographs represent a 3D space on a 2D plane and multi view methods take this concept by converting raw point cloud data into an array of 2D images and apply the convolution operation using CNN to extract features [20].

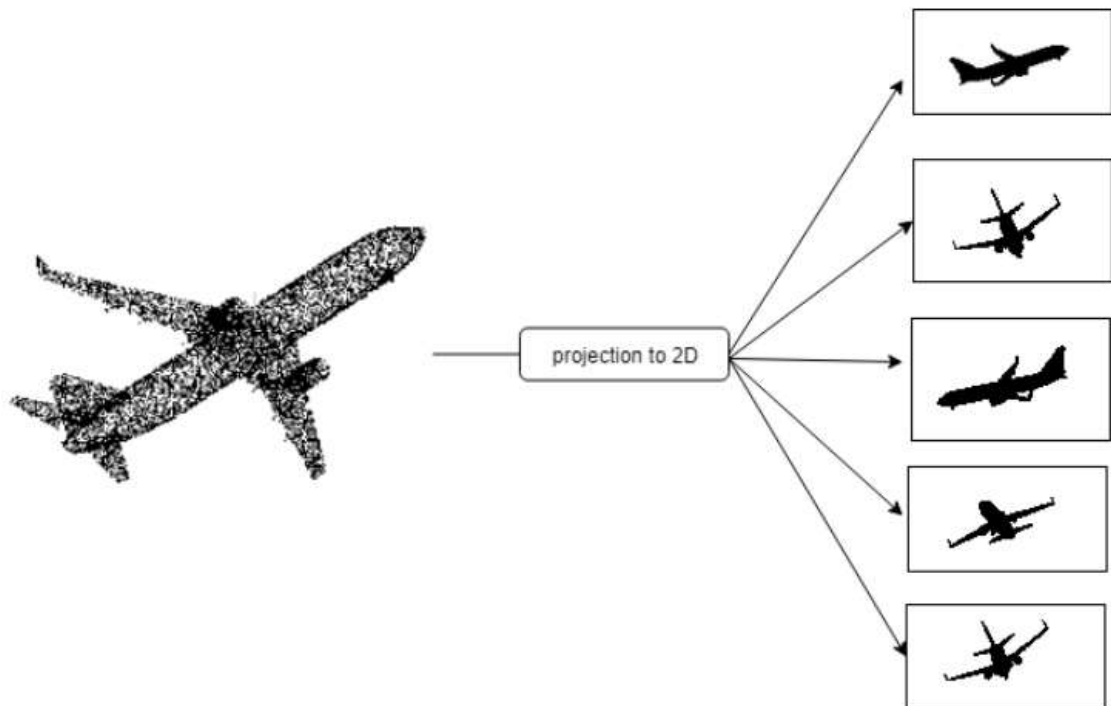


Figure 5: Point cloud to multi-view 2D images

This performed much better than voxel-based methods because 2D based techniques have a much more expansive history of research and it does not produce quantised errors in voxelisation. Even though there is an improvement in CNN methods, it introduces view-dependency meaning that obscure objects can be misrepresented according to chosen viewpoints.

Direct point cloud learning

Despite the challenges of raw point clouds listed earlier, applying deep learning directly onto points without conversion has become popular and different architectures have been developed to become better at extracting features and predictive labelling.

PointNet

In 2017- PointNet was the first architecture to emerge that directly applies deep learning to raw point clouds. It comprises of two networks: the classification network and segmentation network.

By taking n point inputs (x, y, z features), the model applies an input transform which is done by specialised spatial transformer network (STNs) also known as a T-net. T-nets are a mini network that predict transformation matrices and learn how to place input

points into a canonical alignment regardless of its original orientation. This gives transformation invariance as if the input is rotated it will keep consistent representation.

Afterwards, the model uses shared multi-layer perceptron (MLPs) to extract learned features from the canonicalised input points. It is shared as in the weightings of the perceptron are kept the same for each n point.

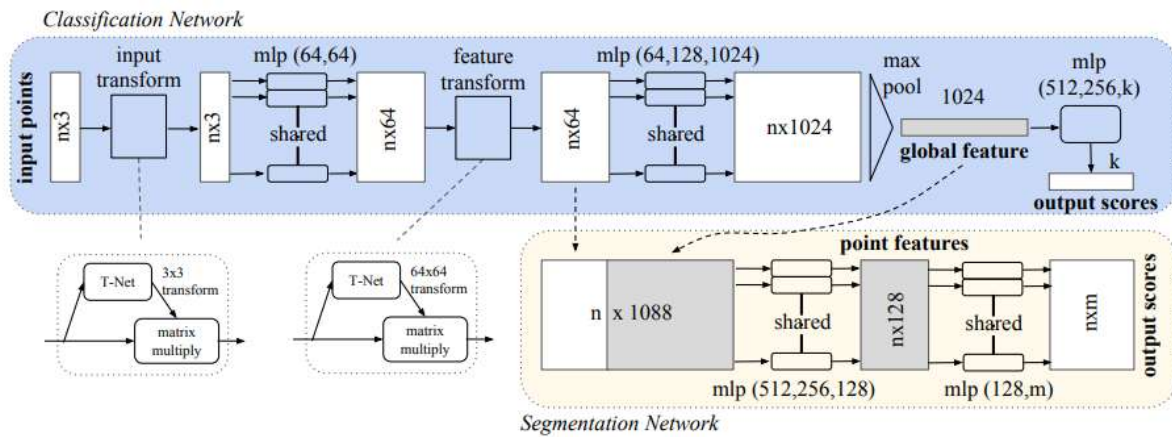


Figure 6: PointNet architecture pipeline

A second T-net then predicts a 64×64 transformation matrix that is applied to the now $n \times 64$ -dimension feature matrix to align it once again to a canonical pose. Further shared MLPs can extract more features.

Finally, using a symmetric function, the feature vector is aggregated using max pooling which makes the new data permutationally invariant. The order of the inputs will not change the output [17].

For classification, this output is given class score, after passing through fully connected layers, which is used to calculate losses in training. As segmentation needs a label for every point, the global vector then concatenates global and local features and outputs scores per point.

Point-net pioneered the way for direct point manipulation however the use of global max pooling means that neighbourhood relationships are lost. Two thin objects such as a fence and a pole may have similar attributes and aren't distinguishable to the PointNet model. Point net pools individual features to a global descriptor without capturing local neighbourhood relationships.

Pointnet++ introduces hierarchical local feature learning through set abstraction layers [22]. This hierarchy allows the model to capture local geometry by progressively downsampling and abstracting local point clouds regions into semantic features. By using feature propagation layers, it can recover the original point cloud resolution from the smaller but more dense representation.

The problem with this model is that it still relies on ball query sampling which can be inconsistent when density varies significantly, a common issue in LiDAR data for urban environments.

Point-transformers

Point transformers do not use any convolution but rather self-attention networks for learning point cloud representations. Specifically vector self-attention as it allows a different weighting per feature channel. Self-attention operators are set operators and use a collection of feature vectors to represent a whole signal or a slice of the signal; this is perfect for point clouds which are just a collection of irregular points in a 3D set. This makes point clouds suitable for set operators as it is an unordered set of coordinates.

The model applies this self-attention only to local neighbourhoods around each point. It does this using a mapping function that consists of an MLP with two linear layers and one non-linear ReLU (This is a mathematical function that when the input is negative, the output becomes 0). The mapping function produces attention weights from the difference of neighbouring feature vectors, which are then used to adjust each feature channel independently.

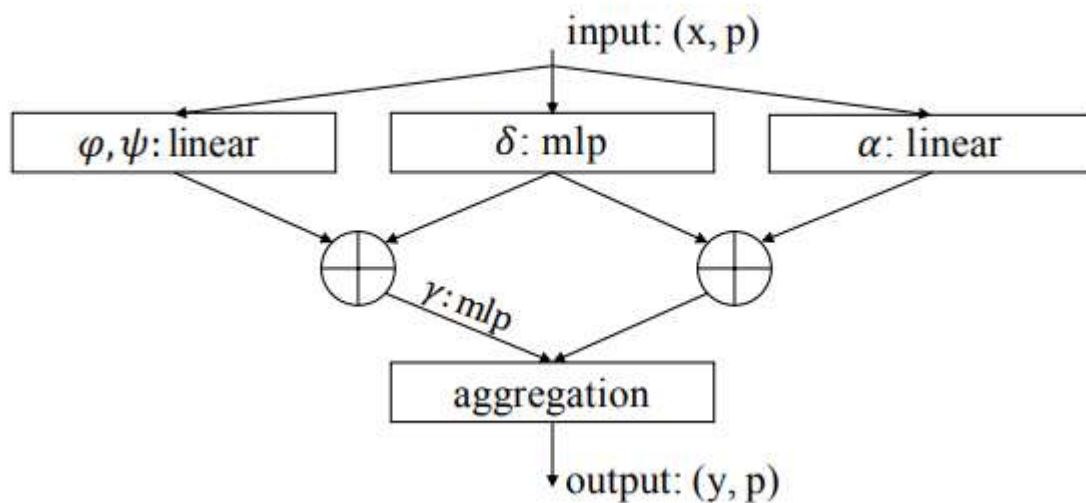


Figure 7: Point transformer layer

The position encoding function gives the vector self-attention information on the relationship of 3D offset between neighbouring points. The encoding function contains an MLP with two linear layers and one non-linear ReLU. This function is essential for both the attention generation branch and feature transformation branch because they both rely on spatial information to aggregate and transform features. Because both functions use MLPs, their parameters are learned during training rather than being fixed which allows the model to discover the spatial relationships between points.

Point transformers achieve very strong performance however the self-attention networks are much more complex and require much more computation. It would

require significant downsampling to run and is not necessary for the scale for this project.

KPConv

KPConv uses a convolution operator directly on points to extract learned features [23]. It does this by using kernel points within a kernel radius and applying weightings depending on the distance between neighbouring points and kernel points. Kernel weights are multiplied with influence weights to get a new feature vector at each point. The flexibility of kernel placement in continuous 3D space allows KPConv to capture precise local structures and by stacking multiple layers with progressive subsampling, the network can learn hierarchical features. There are two variants, rigid and deformable kernels. Rigid kernels have a set radius whilst deformable radius learn kernel point positions which adapt their sampling positions.

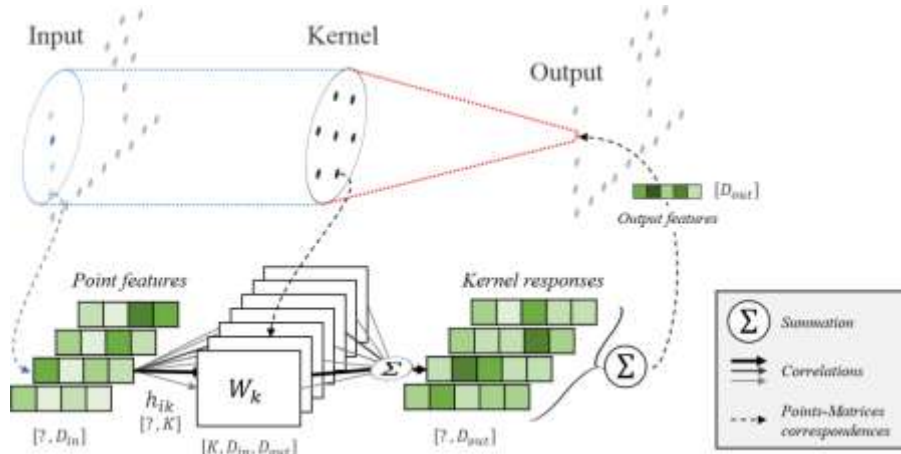


Figure 8: KPConv on a 2D image

KPConv unfortunately is very sensitive to subsampling parameters and the choice of grid cell size that controls the point density if not chosen appropriately can discard fine geometric detail. The deformable variant will introduce more instability and can result in the model not converging as kernel points can drift outside of influence ranges creating null gradients that require regularisation to manage (see theoretical background). KPConv cannot dynamically shift weights on class importance like transformer-based approaches which may be a bottleneck in its ability to handle diverse urban landscapes.

KPConvX is an extension of the architecture that incorporates kernel attention, inverted bottlenecks and nearest-kernel assignment [24]. Although more computationally efficient than KPConv, the hyper parameters make tuning significantly harder to interpret and difficulty in what changes would result in improvements to the model.

Given the limitations across all the previous methods and what is most suitable for the scale and objectives trying to be completed, KPConv is chosen as the most suitable. KPConv addresses the key limitations of the preceding methods, it operates directly on

points without needing voxelisation, significant downsampling due to memory limits and uses local neighbourhood convolution for the best attention at this neighbourhood scale. KPConv replaces ball query sampling found in PointNet++ with radius-based neighbourhoods providing more consistent feature extraction for non-uniform point densities. The deformable variant is especially applicable where objects vary in shape and scale significantly [25]. KPConv already has existing benchmarks on the Toronto3D database [26] allowing comparison of optimised outcomes with published results.

Theoretical Background

The KPConv operation

Most of the architectural background can be found from the conference paper given in 2019 [23].

KPConv uses the convolution operation (1) to compute new features at point x .

$$(\mathcal{F} * g)(x) = \sum_{x_i \in \mathcal{N}_x} g(x_i - x) f_i \quad (1)$$

x_i belongs to point cloud P as $P \in \mathbb{R}^{N \times 3}$. There are N points in the point cloud each with 3 input features as x , y and z coordinates.

f_i belongs to feature vector $\mathcal{F}$ as $\mathcal{F} \in \mathbb{R}^{N \times D}$. For N points corresponds to D learned features.

The chosen radius (2) ensures that when defining neighbours to point x_i there will be a robustness for varying neighbour densities.

$$\mathcal{N}_x = \{x_i \in P \mid |x_i - x| \leq r\} \quad (2)$$

The neighbourhood of point x , $\mathcal{N}_x$ is defined as all points in the point cloud whose distance magnitude between chosen point and all other points are less than the chosen radius.

Function g helps the convolution operation learn useful operations. Neighbour points were defined earlier as $x_i - x$ and this is now called y_i . This domain is a ball described by:

$$\mathcal{B}_r^3 = \{y \in \mathbb{R}^3 \mid ||y|| \leq r\} \quad (3)$$

The neighbour points were defined in (2) and is referred to as $y = x_i - x$ in (3).

The g function must apply different weights to different areas inside this domain.

There is a set of kernel points within a 3D ball of radius r from index 0 to $K-1$ (4). Each kernel point has a corresponding weight matrix with a size of input and output features $D_{in} \times D_{out}$ (5).

$$\{x_k \mid k < K\} \subset \mathbb{B}_r^3 \quad (4)$$

$$\{W_k \mid k < K\} \subset \mathbb{R}^{D_{in} \times D_{out}} \quad (5)$$

The kernel function g (6) is shown as:

$$g(y_i) = \sum_{k < K} h(y_i, \tilde{x}_k) W_k \quad (6)$$

By scaling weight matrices W_k with a distance-based correlation h between the local neighbourhood point y_i and kernel point x_k and summing all the scaled kernel weight matrices, the function can be computed.

Function h measures how close neighbour y_i is to kernel point x_k (7). This is then normalised by the influence distance σ . Negative values are given value 0 to create a weighting from 1 to 0 based on the normalised distance.

$$h(y_i, \tilde{x}_k) = \max\left(0, 1 - \frac{\|y_i - \tilde{x}_k\|}{\sigma}\right) \quad (7)$$

There are two KPConv variants, rigid kernels with a fixed geometry or deformable kernels with kernel points adapting to local point cloud geometry.

In rigid kernels, kernel points act repulsively to each other and are constrained in the ball with an attractive force. There will always be a kernel point in the centre, and the surrounding points are scaled to an average radius of 1.5σ . This is done so the influence distance slightly overlaps so that there will be a decent coverage of influence for the kernel points.

The kernel positions can be learned as function g is differentiable with respect to $\tilde{x}_k$. Within the feature map, the network will learn to dynamically shift kernel points based on the local features rather than keeping the same grid pattern. Therefore, a deformable kernel can be defined as (8):

$$(\mathcal{F} * g)(x) = \sum_{x_i \in \mathcal{N}_x} g_{\text{deform}}(x - x_i, \Delta(x)) f_i \quad (8)$$

Applying a deformable kernel to a point cloud neighbourhood gives the equation (9):

$$g_{\text{deform}}(y_i, \Delta(x)) = \sum_{k < K} h(y_i, \widetilde{x}_k + \Delta_k(x)) W_k \quad (9)$$

The offsets $\Delta_k(x)$ are produced by the output of a rigid KPConv mapping the D_{in} input features to 3K values (Offset vectors each with an x, y, z coordinate). When training, the rigid kernel will learn to generate offset shifts, and the deformable kernel will learn to generate output features using the shifted points. A fitting regularisation loss (11) must be applied as kernel points can be lost by the network when there are no neighbours in their influence range leading to a null $\Delta_k(x)$. This loss punishes the distance between a kernel point and its closest input neighbour. There is also a repulsive regularisation loss (12) between all kernel points when there is an overlap of influence so that they don't all collapse. Together as a general regularisation loss (10), both losses make sure that kernel points stay spread out with good distribution whilst being anchored to local geometry.

$$L_{\text{reg}} = \sum_x (L_{\text{fit}}(x) + L_{\text{rep}}(x)) \quad (10)$$

$$\mathcal{L}_{\text{fit}}(x) = \sum_{k < K} \min_{y_i} \left(\frac{\|y_i - (\widetilde{x}_k + \Delta_k(x))\|}{\sigma} \right)^2 \quad (11)$$

$$\mathcal{L}_{\text{rep}}(x) = \sum_{k < K} \sum_{l \neq k} h(\widetilde{x}_k + \Delta_k(x), \widetilde{x}_l + \Delta_l(x))^2 \quad (12)$$

Intersection over union and the Lovász-Softmax loss

IoU is the primary metric to measure performance of deep learning models due to its effectiveness for spatial description. It is the percentage of overlap between the overlap area and over the total combined area. If the model predicts a larger coverage area, then the accuracy metric would be 100% accurate even though it has overcompensated the actual object however IoU would account for this extra area outside the truth. Deep learning models are trained to minimise cross entropy loss, how the predicted probability distribution matches the ground truth. The Jaccard index (IoU) is shown in (13) where y^* is the ground truth label vector and $\tilde{y}$ is the vector of predicted labels for class c .

$$J_c = \frac{|\{y^* = c\} \cap \{\tilde{y} = c\}|}{|\{y^* = c\} \cup \{\tilde{y} = c\}|} \quad (13)$$

Cross entropy penalises per-pixel however IoU is a global mask for a collective structure so therefore there are different goals in training and evaluation. This means that

optimising the Jaccard loss (14) is more desirable to optimise semantic segmentation IoU.

$$\Delta J_c(y^*, \hat{y}) = 1 - J_c(y^*, \hat{y}) \quad (14)$$

The Jaccard loss should be minimised in training to get best IoU values. However, since this loss is discrete, it can't be used directly in training; it is non differentiable. Pixels either belong to a class or don't, there is no gradient to operate from. [27] considers extensions based on submodular analysis.

For multiclass segmentation, the network outputs a probability for each pixel belonging to a specific class via a softmax unit (15).

$$f_i(c) = \frac{e^{F_i(c)}}{\sum_{c' \in \mathcal{C}} e^{F_i(c')}}, \quad \forall i \in [1, p], \forall c \in \mathcal{C} \quad (15)$$

f_i is the probability of a pixel to be in class c . F_i is the unnormalised scores of the network.

Using class probabilities defined by (15), a vector of pixel errors can be defined as (16).

$$m_i(c) = \begin{cases} 1 - f_i(c) & \text{if } c = y_i^* \\ f_i(c) & \text{otherwise} \end{cases} \quad (16)$$

The error shows pixel error or if the pixel is in the correct class, how far the probability is from a confidence of 1.

This can be fed into Lovász extension (17) and averaged across all classes (18).

$$\Delta: m \in \mathbb{R}^{\mathbb{P}} \mapsto \sum_{i=1}^p m_i g_i(m) \quad (17)$$

with $g_i(m) = \Delta(\pi_1, \dots, \pi_i) - \Delta(\pi_1, \dots, \pi_i - 1)$, (p being number of pixels in the batch)

$$Loss(f) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \bar{\Delta} J_c(m(c)) \quad (18)$$

By constructing continuous error vectors, it is a differentiable substitute for Jaccard loss by approximating it allowing for gradient-based training. This directly targets IoU during training unlike cross-entropy loss.

Network layers

Point clouds are inherently non-uniform as discussed earlier, they have varying point densities. This can pose computational challenges as dense regions have far more points to process slowing the network and kernel behaviour will change depending on

different areas in the cloud. To combat this subsampling is required to ensure spatial consistency. In the original KPConv paper [23], grid subsampling is chosen to divide 3D space into cells and replaces all points in this cell with an average position (barycentre), allowing equal representation per cell.

Since this architecture has multiple layers with different scales, it needs pooling layers to reduce the number of points per layer so that each subsequent layer can operate at a coarser spatial resolution which allows the network to progressively capture larger geometric structures whilst reducing computational cost. This is done by doubling the cell size at every pooling layer whilst increasing the receptive field of KPConv. The features are then obtained by a KPConv operation allowing the network to learn how to aggregate features.

The convolution layer takes three inputs: point positions with 3D coordinates (19), corresponding features at each point (20) and a matrix storing neighbourhood indices at each location (21).

$$\mathcal{P} \in \mathbb{R}^{N \times 3} \quad (19)$$

$$F \in \mathbb{R}^{N \times \mathbb{D}_{in}} \quad (20)$$

$$\mathcal{N} \in [1, N]^{N' \times n_{max}} \quad (21)$$

N' is the number of places where neighbourhoods are computed. The neighbourhood matrix is the size of the biggest neighbourhood n_{max} however most neighbourhoods are smaller than this. This means there will be unused elements called shadow neighbours that will be ignored during computation.

The parameters in each layer j are derived from the cell size dl_j .

Influence distance is set to (22).

$$\sigma_j = \Sigma \times dl_j \quad (22)$$

The rigid KPConv convolution radius is set automatically to $2.5\sigma_j$ as the average kernel point radius is $1.5\sigma_j$. The deformable convolution radius is chosen as (23).

$$r_j = \rho \times dl_j \quad (23)$$

KPConv has two types of architecture for classification and segmentation, KP-CNN and KP-FCNN. KP-CNN is structured as a 5-layer classification network, each layer is built around a pair of bottleneck residual blocks. Substituting the standard image convolution with a KPConv operation and with batch normalisation, it follows the ResNet design. To reduce spatial resolution progressively, the first block of each layer uses strided convolution except from the initial layer. When the final layer is reached, a global average pooling operation aggregates learned features that are fed into fully connected SoftMax layers to produce class predictions. Deformable kernels if applicable will only be introduced in the final 5 KPConv blocks.

KP-FCNN is fully convolutional designed for segmentation. This network needs to recover spatial information through the decoder as it needs to label every point in the scene. It has the same encoder as KP-CNN however adds a decoder that progressively up samples through nearest upsampling to bring features to full resolution. The decoder includes skip-links that pass features from the encoder through to the corresponding decoder layer to preserve geometric detail that would otherwise be lost during downsampling. The features are concatenated to the upsampled ones and processed via unary convolution which is synonymous to a shared MLP in PointNet.

Methodology and Results

There are many datasets available to train deep learning models and considering the scope of the project as a small neighbourhood, Toronto3D makes the most sense for semantic segmentation [1]. The original KPConv architecture can be found on GitHub <https://github.com/HuguesTHOMAS/KPConv>. It was run on Google Colab however local backups were maintained just in case of accidental loss. The KPConv is open source so no IP infringed upon. Within this architecture there is a training script for Toronto3D and includes class variables that can be tweaked such as number of kernel points, input radius, convolution radius, number of input features etc.

There are two scripts that were mainly relevant and used for modification and that was the model trainer script and the Toronto3D trainer script. The model trainer script handles the training and evaluation pipeline. The Toronto3D training script prepares the Toronto3D data, sets up the environment for training and handles all training parameters.

Running the Toronto3D training script outputs metrics such as offset loss, accuracy, step loss and number of epochs. Using MATLAB, the IoU and the per class IoUs were plotted. The per class IoU was measured to see how different models would respond to different classes.

The Toronto dataset has dense point clouds with semantic annotations for 8 classes however class distribution is imbalanced, reducing representation for minority classes.

First was to run the initial settings as a rigid and deformable kernel to compare the differences between the two in performance. This was run at 50 epochs because this was deemed long enough to see general trends in the model without taking too much computational power and time however, there is a trade-off that the models can be more volatile without settling first.

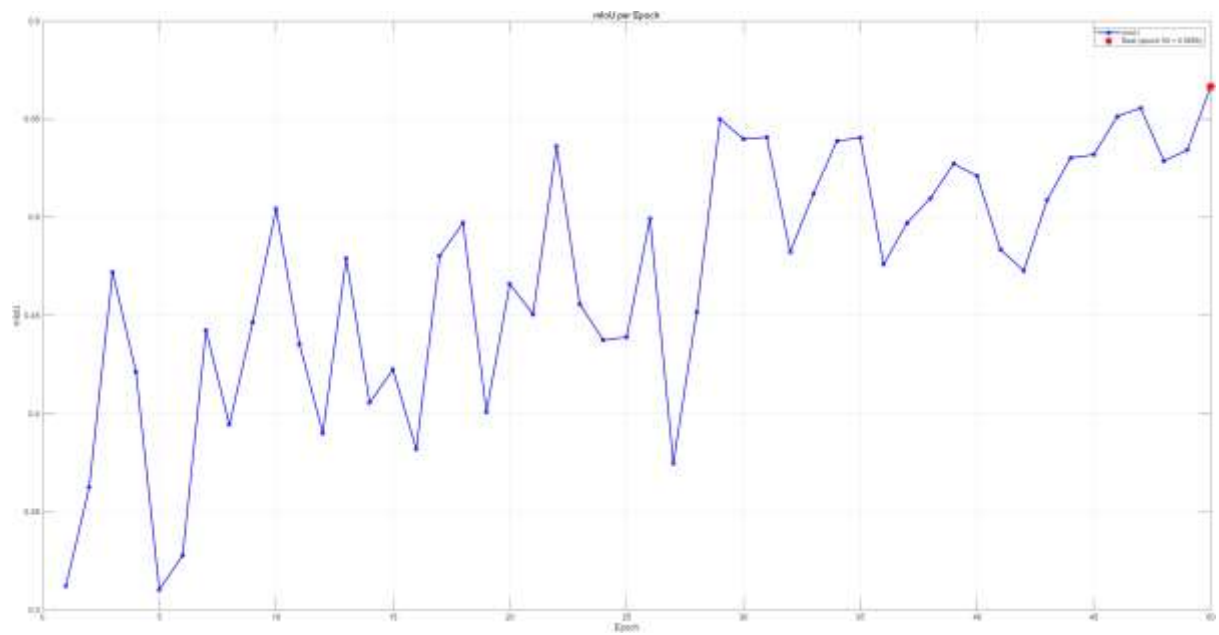


Figure 9: Mean IoU per epoch for the rigid kernel

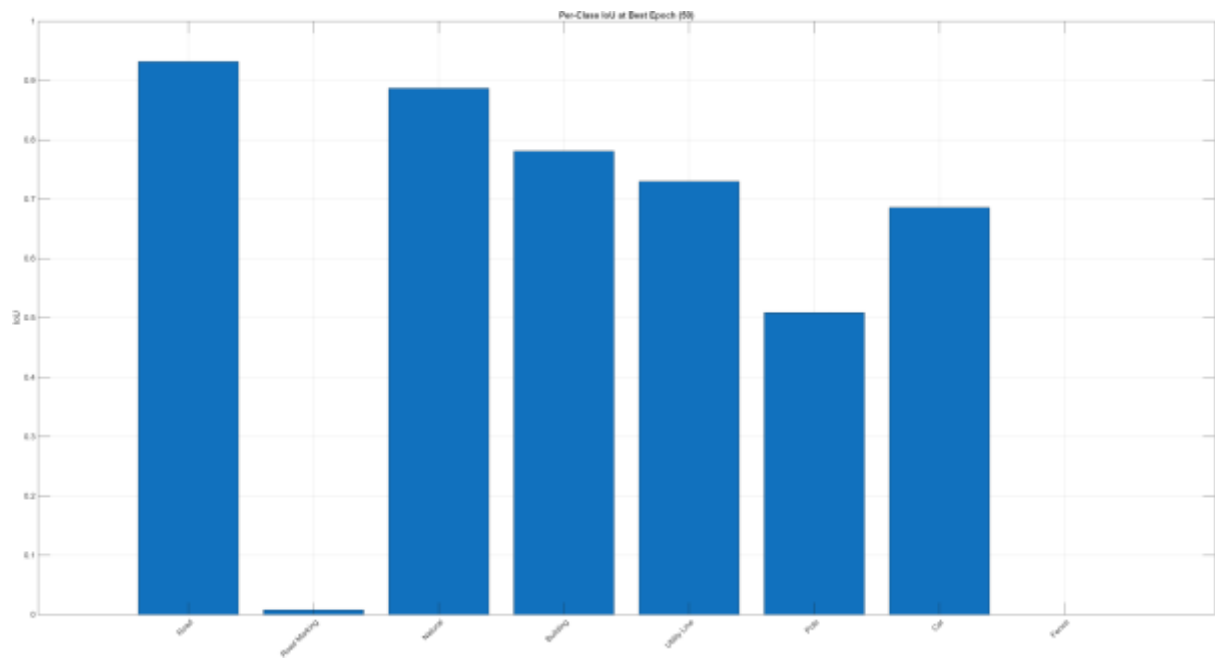


Figure 10: Per-class IoU at highest performing Epoch for the rigid kernel

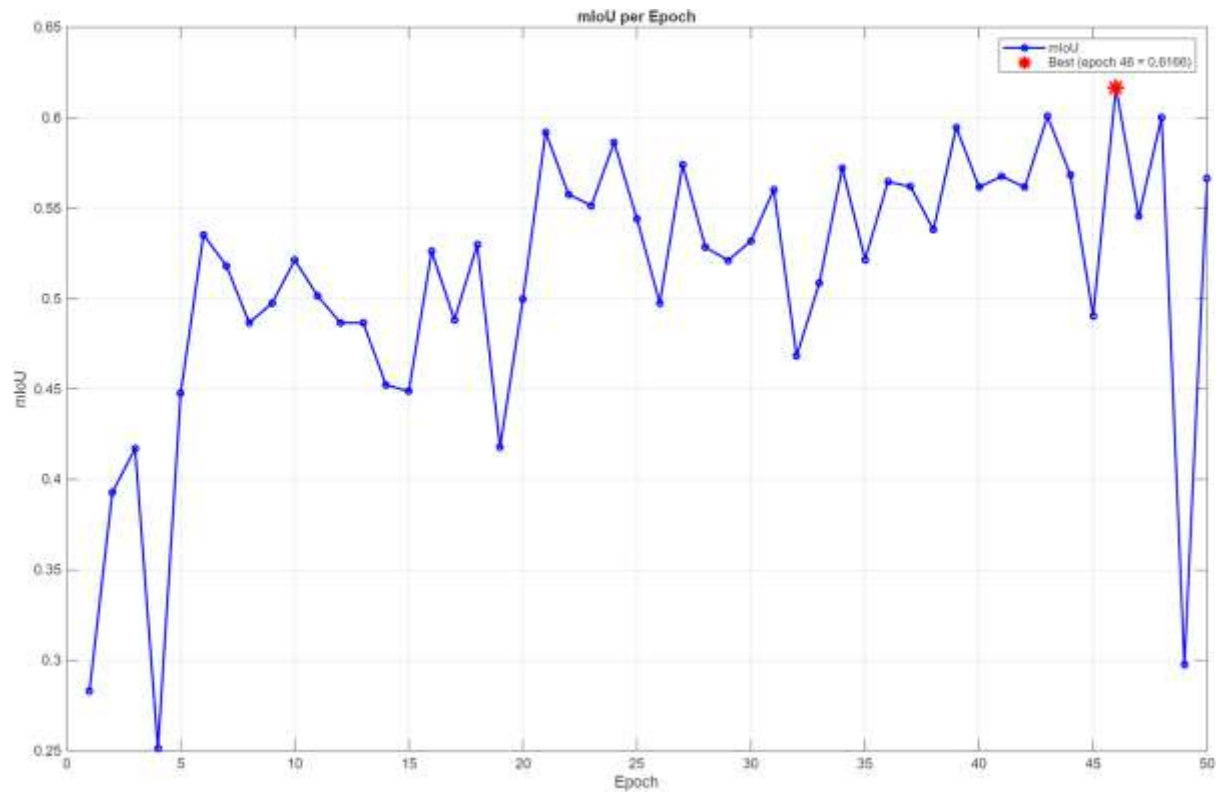


Figure 11: Mean IoU per epoch for the deformable kernel

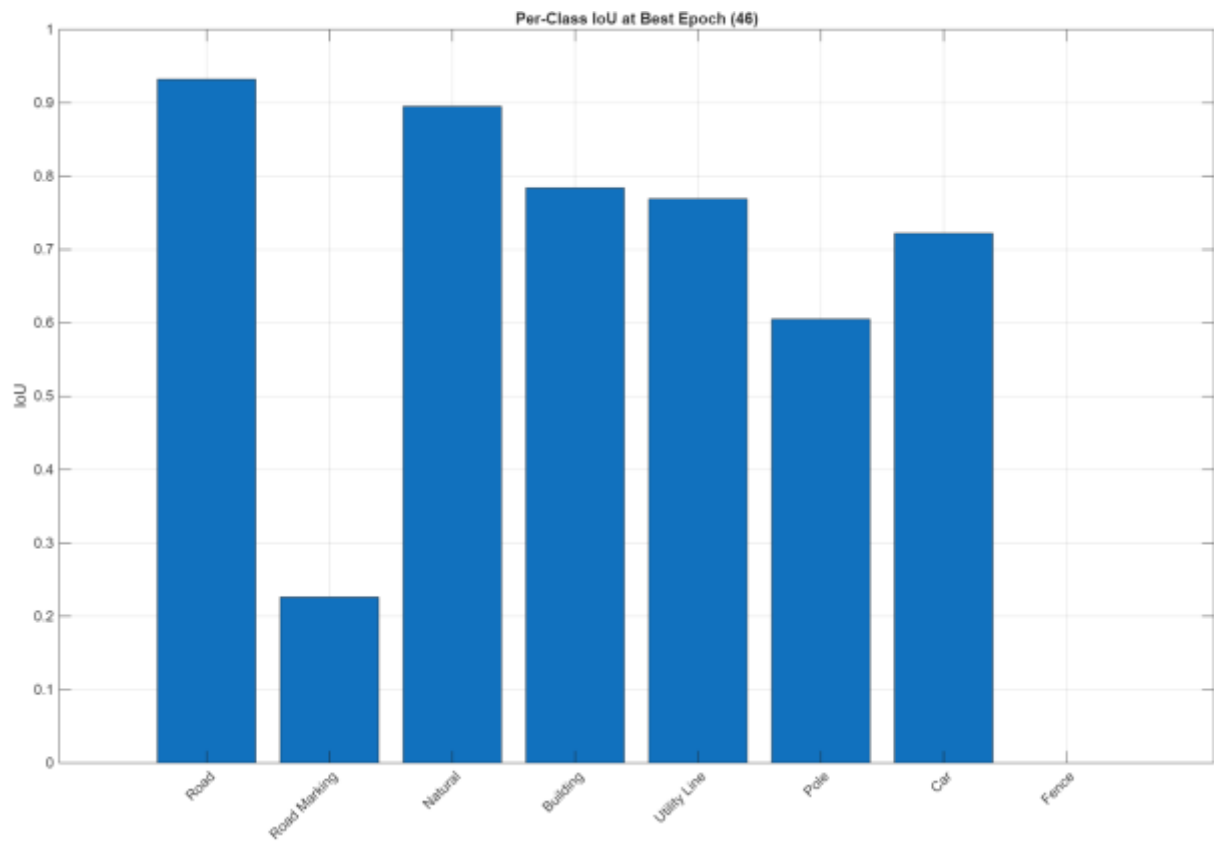


Figure 12: Per-class IoU at highest performing Epoch for the deformable kernel

The rigid performance is poor with the model only reaching a maximum mIoU of 0.57. Figure 12 shows that it is weak at detecting road marking and fences. There is a lot of

improvement that was made through the iterations. Through reflection, the iterative approach allows a constant workflow of improvement however doesn't isolate specific parameters to analyse for direct impact to the model. Comparing it to the deformable kernel which had a maximum IoU of 0.62 which is already a considerable performance boost. This network was able to detect road markings although not to a high standard and it still is not able to detect any fencing. It was expected to outperform rigid kernels due to its learnable shifts in kernel positioning so was much better at finding longer, thinner objects so it indicates it has more precision.

The following modifications to the code/parameters are done in iterations, building from the last iteration to the final model to find the optimal settings for semantic segmentation.

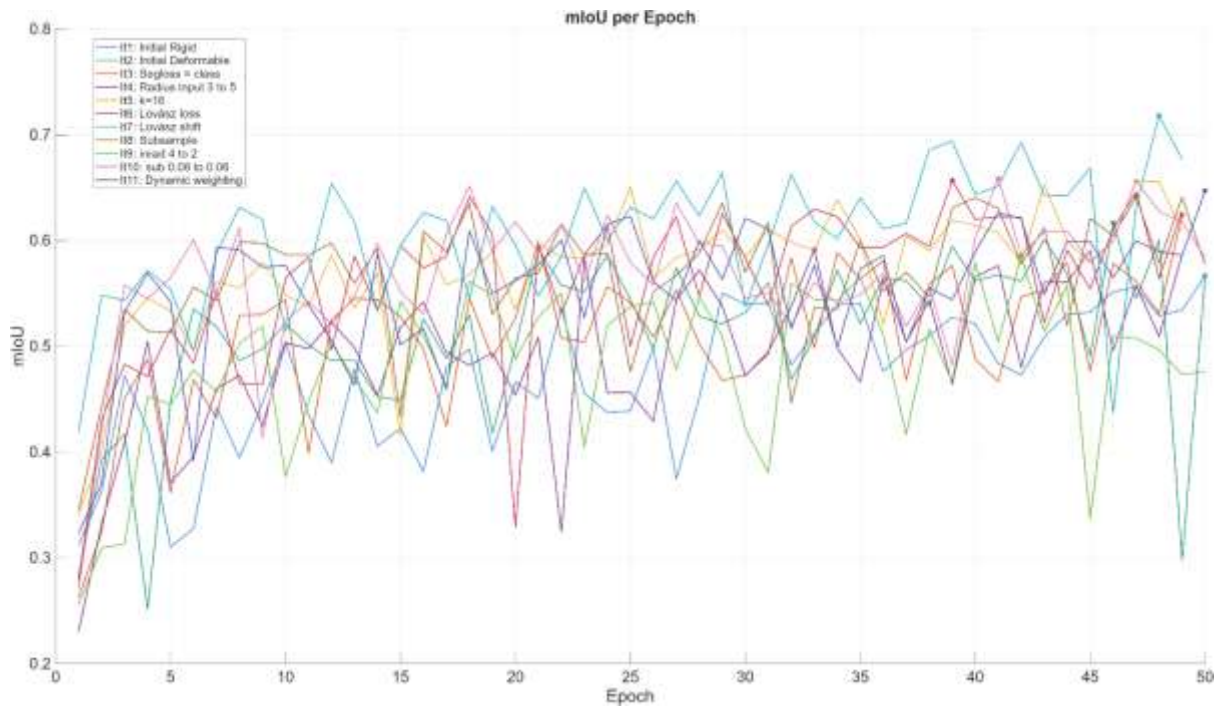


Figure 13: mIoU per epoch through iterations

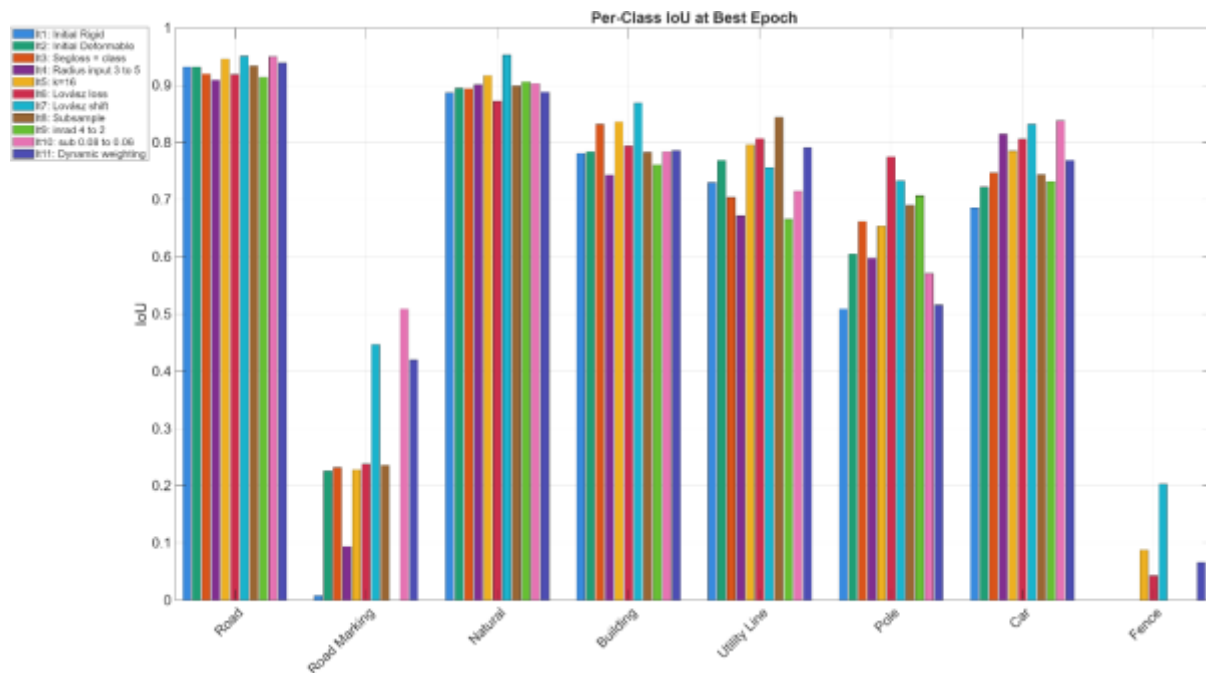


Figure 14: Per-class IoU at best epoch comparison through iterations

Changing the segmentation loss balance to class mode handles the class imbalance in the dataset. When in this class mode, the loss function reweights all the points so that less frequent classes get more influence per point. This means all classes contribute equally and therefore the model can properly detect rarer classes. The best mIoU was 0.62 so there wasn't much improvement, even though rarer classes now contribute more to the loss, they appear much less frequently in the batches so won't give much change.

The input sphere radius was increased from 3m to 5m. This is essentially allowing the model to see more data from the dataset at a time for processing. This had a negative effect on the performance, with the best mIoU being 0.59, as it ends up attenuating local geometry. This negative performance is because the model sees sparser representation with same kernel points. Subsampling becomes more aggressive losing more local features as sampling density decreases which can hurt edge detection for example.

Adding another kernel point from 15 to 16 was done to counteract the lower sampling density. It is clear to see that despite the higher computational cost, the model could finally observe fences in the dataset. From the theoretical background, more kernel points will lead to a better understanding of the local geometry but increasing over 16 kernel points was found to be redundant when training as the increase in performance was not proportionate with the model failing to converge. The best mIoU was 0.65 so a considerable improvement and for the first time the model can detect fencing.

Implementing a Lovász-Softmax loss to the trainer module and train_Toronto3D scripts helped optimise the intersection over union. This didn't affect the model much from the module implementation from the previous iteration. This is because the Lovász loss isn't effective during the latter stages of training, linking back to the theoretical

background as model confidence grows the loss gradient becomes weaker and will be very noisy for the unconfident classes as it is applied to all classes. The mIoU stayed the same with similar results for per-class IoUs.

To fix this, the trainer script was adjusted so that the loss function changes throughout training, instead of a fixed loss weight it will shift from Lovász to cross entropy loss. Also, further modifications were made to handle unclassified points and ensure label indices are correctly mapped to values the loss function expects. This improved the model to a best mIoU of 0.72, this shift from Lovász loss to cross entropy loss gives the best balance to optimise the mIoU values.

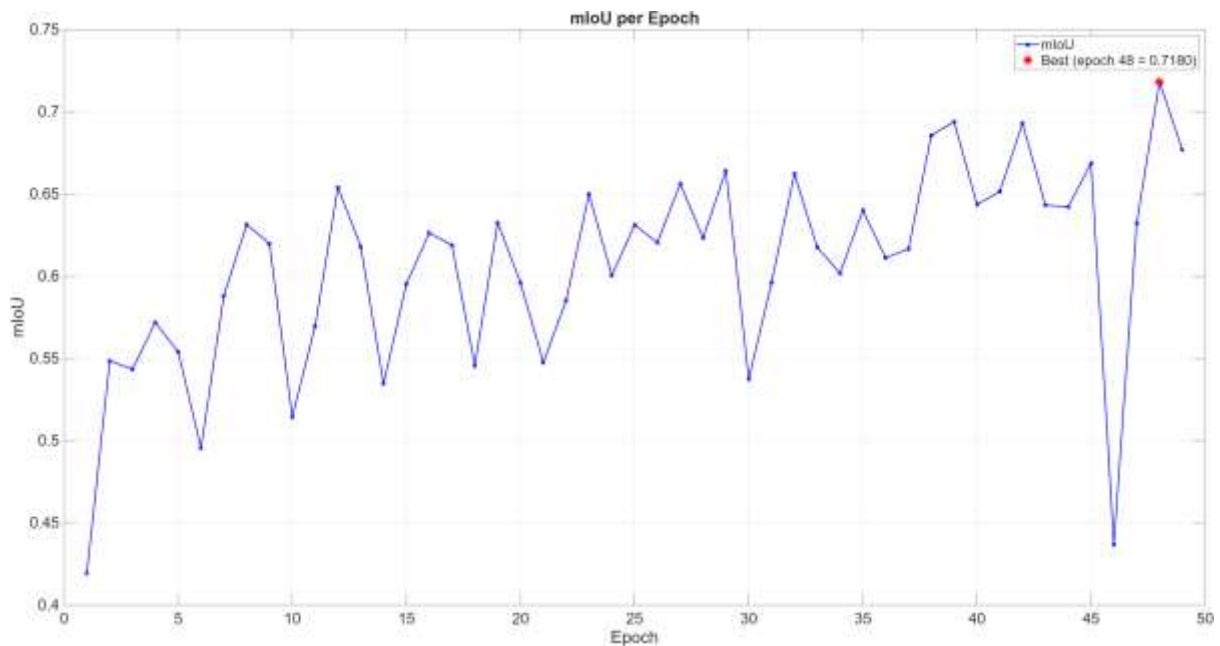


Figure 15: Mean IoU per epoch for the shifting Lovász loss

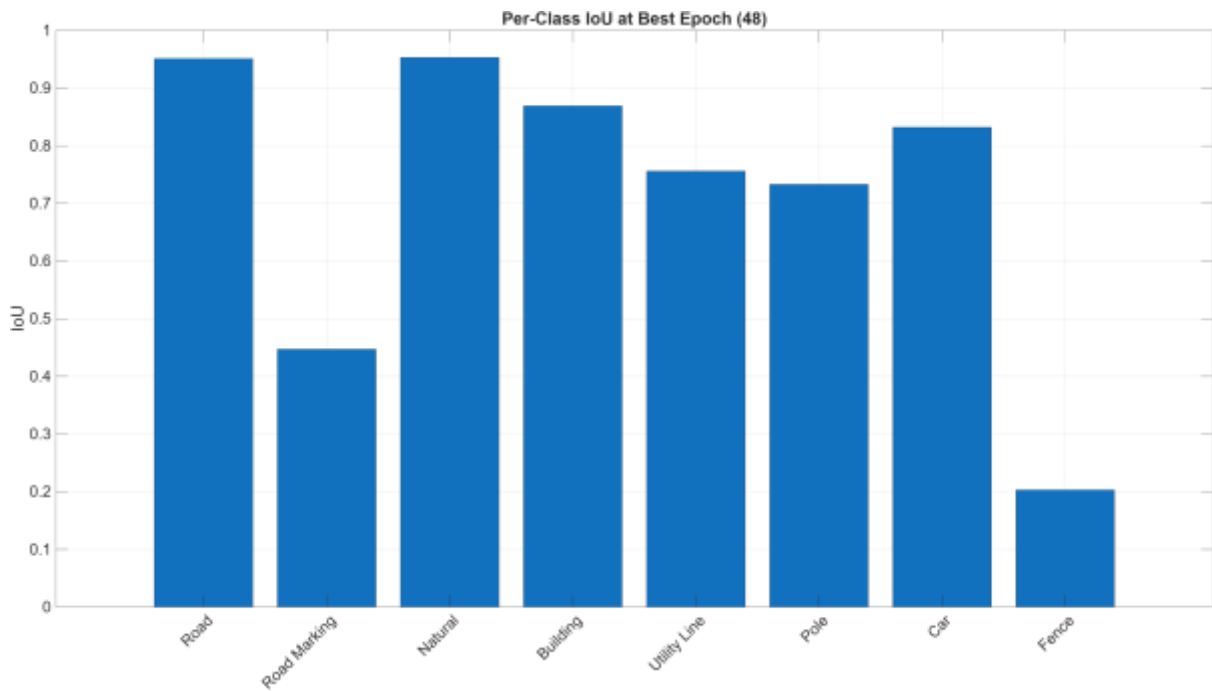


Figure 16: Per-class IoU at highest performing Epoch for shifting Lovász loss

Computational parameters were increased resulting in slower simulation times. The subsample was decreased from 0.08m to 0.06m, input sphere radius decreased from 5m to 4m, augmentation scale minimum and maximum widened with the augmentation noise halved. This higher computation gave a better mIoU of 0.73. Finer subsampling means downsampling is less aggressive and more detail is preserved. Wider augmentation allows for improved generalisation as the model can handle varying distances between points better. Reducing augmentation noise means that the greater detail that has been gained will not be ruined by the extra noise and can use these details for evaluation.

Then the input sphere radius was reduced to 2m with first subsampling increasing from 0.06 to 0.1. This showed how important first subsampling is as the model was unable to detect fencing or road markings with a best mIoU of 0.57.

First subsample was then decreased from 0.1 to 0.06 as to see these two classes, input radius changed from 4m to 3m. The best mIoU was 0.65 with no fence detection. This is because they are very thin structures and by reducing the input sphere, the model may not see the whole picture and points can become ambiguous.

Writing dynamic class weightings to the Toronto3D training script was done so that IoU values were inverted at the end of validation and chosen to be the loss values so that low confidence classes could improve at a faster rate whilst confident classes did not change very much as they are already very confident in the prediction. The best mIoU was 0.65.

The best performing models were run at 200 epochs to get ready for testing. This was determined from figure 13 which shows the comparison of the first 50 epochs. These

were chosen to be iterations 7 (Lovász-shift), 10 (0.06 subsampling and 3m input radius) and 11 (Adaptive classes) as they had the steadiest improvement in mIoU.

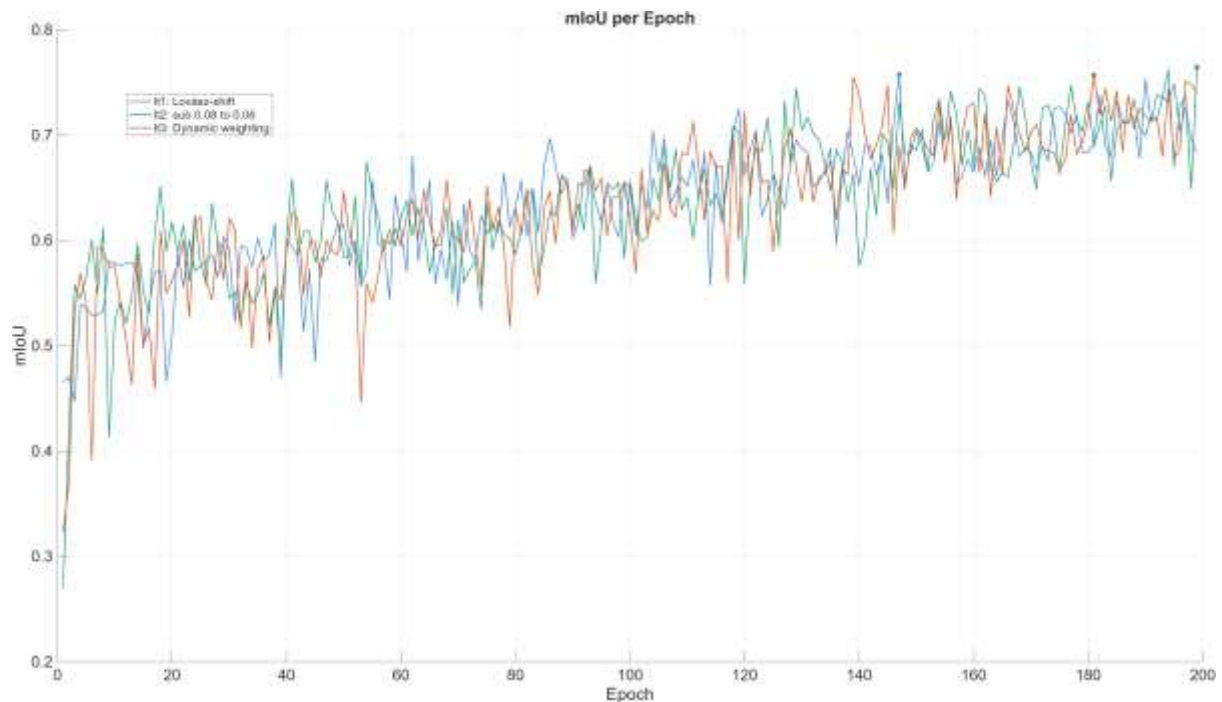


Figure 17: 200 epochs of best models with mIoU

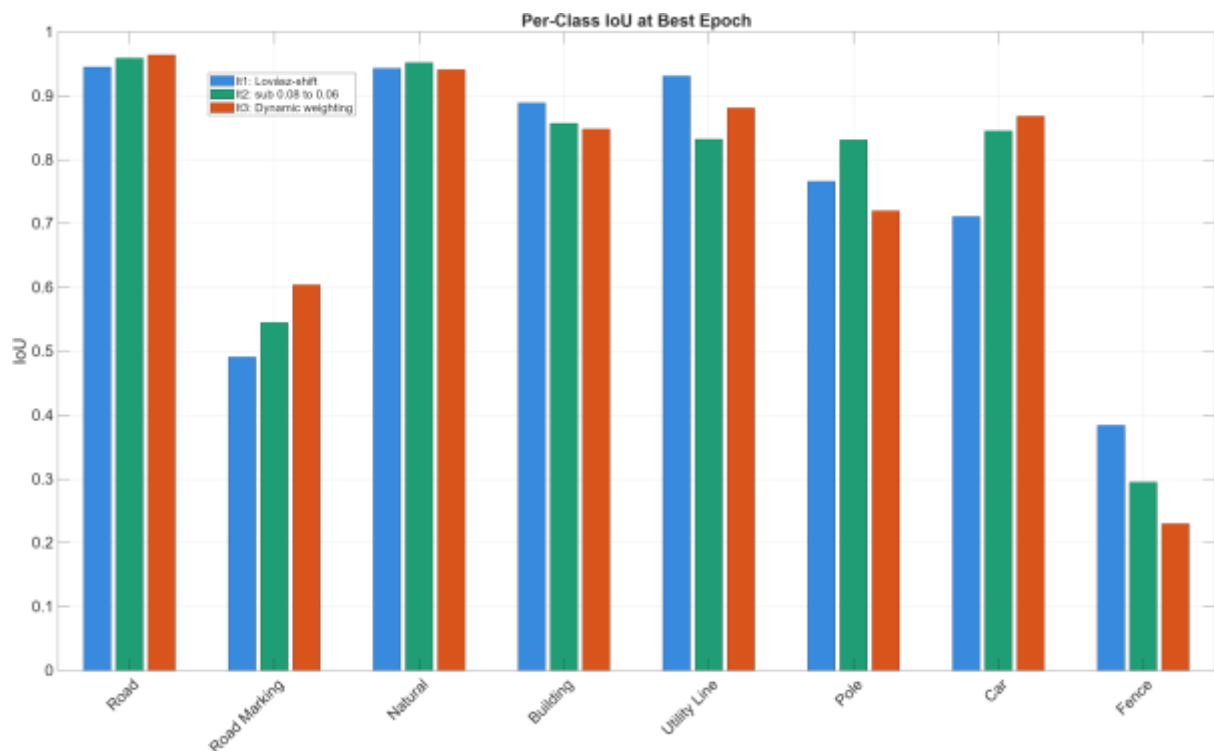


Figure 18: Per-Class IoU at best epochs for best models.

Road and natural object detection was no issue, with Lovász-shift being the best performing iteration standing at 0.76 mIoU. It was also best at predicting fences and poles whilst the dynamic weighting model was best at detecting cars and road

markings. The lower subsampling model gives a good balance between the two models. This shows that by adding a dynamic weighting class to the model, it is able to change priorities on what you wish to detect. If the model is needed for traffic planning then dynamic weighting can be implemented however if the architectural needs of the environment are more important for capture then that class won't be needed.

The lower values for road marking and fences can also be because there are fewer points in the dataset, the model has less information to use for training therefore will naturally just be worse at detection.

From testing the models using a testing script, the prediction ply files can be obtained and put in CloudCompare, a software for visualising these files.

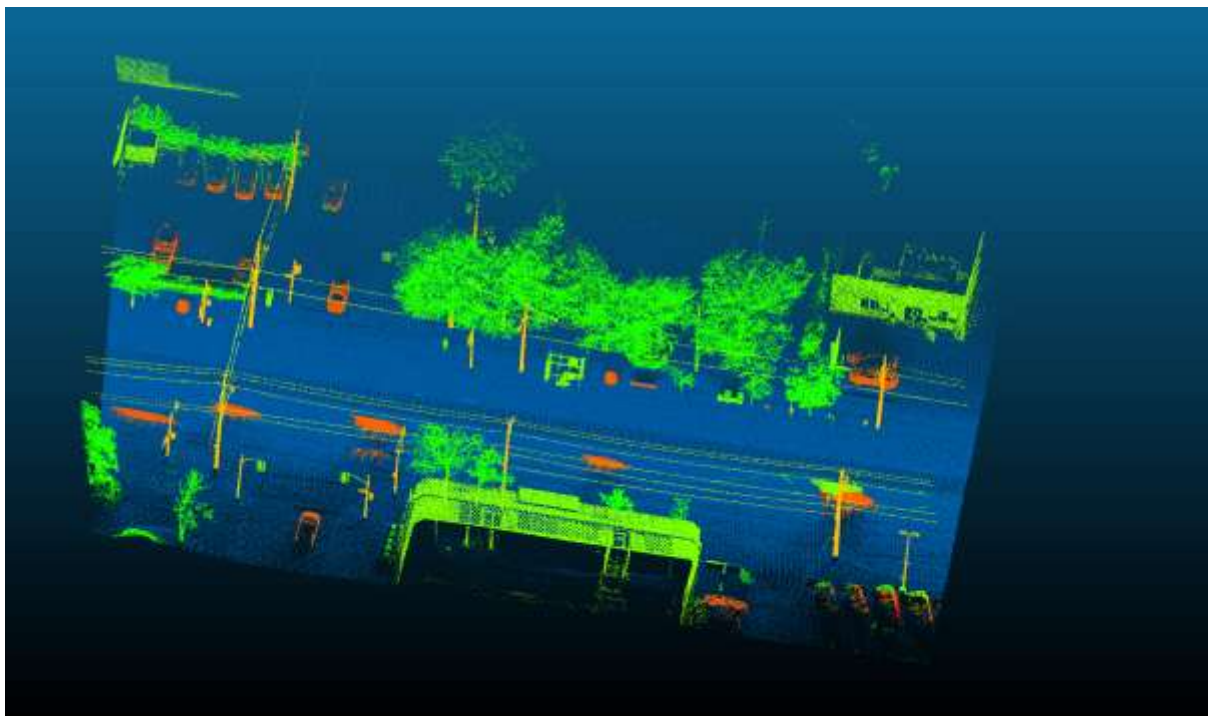


Figure 19: Initial KPConv Rigid prediction map

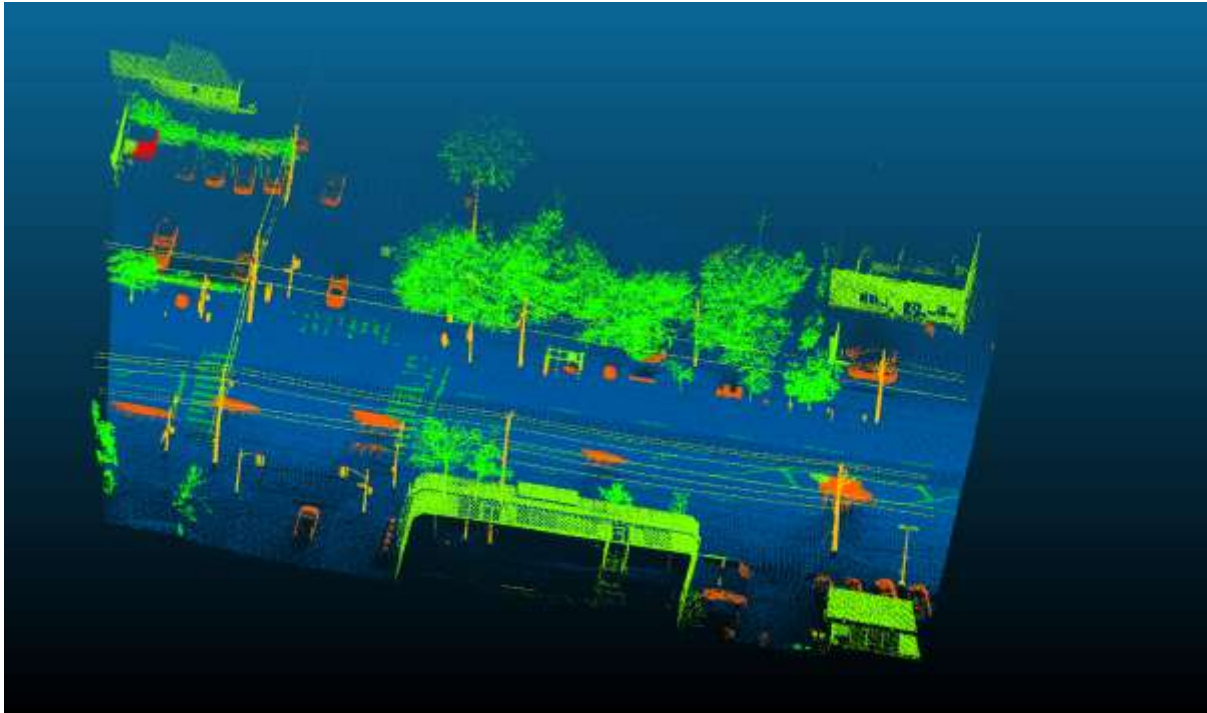


Figure 20: 0.06 subsample model prediction map

By comparing the two you can see that the road markings are clearer and in figure 20 it has identified in the bottom right corner a building that figure 19 misses completely. Figure 20 shows better segmentation capabilities for example by looking at a complicated structure of the bus stop, it can still segment it showing the stand is separate from the bench and the advertising. Some cars in figure 19 are incorrectly segmented and have conflicting points whilst figure 20 can identify all the points belonging to the car class.

Conclusion

This project successfully optimised a KPConv deep learning model for semantic segmentation of LiDAR point cloud data on the Toronto3D dataset, improving the mIoU from a rigid kernel baseline of 0.57 to a peak of 0.76 through iterative modification of training scripts. Key improvements included the use of deformable kernels, increasing kernel points and implementing a shifting Lovász-Softmax loss that was used to directly optimise IoU during training before transitioning to cross-entropy loss for stability.

The three best models demonstrated that model configuration can be tailored for specific demands rather than a global solution. Creating a cloud map in CloudCompare highlights the improvement in semantic segmentation and these maps can be used in real-world applications to create a registry of a dense urban environment.

The scope was limited to a single neighbourhood dataset meaning generalisation for larger datasets being untested. Future work could explore combining multiple architectures to handle broader labelling spaces and object-level classification. A real urban environment contains far more object classes than those covered by Toronto3D,

so extending the model to recognise and assign labels to previously unseen classes would be a valuable direction.

References

- [1] N. Q. L. M. Y. L. J. D. G. C. K. Y. J. L. Weikai Tan, “Toronto-3D: A Large-Scale Mobile LiDAR Dataset for Semantic Segmentation of Urban Roadways,” in *Conference on Computer Vision and Pattern Recognition*, Toronto, 2020.
- [2] Z. Ballouch, “A Prior Level Fusion Approach for the Semantic Segmentation of 3D Point Clouds Using Deep Learning,” *Remote sens*, vol. 14, no. 14, 2022.
- [3] A. A. Majrashi, “Participatory Urban Digital Twins with Fairness Aware Optimization for Equitable Quality of Life Improvements in Multi Objective Planning Contexts,” *International Journal of Environmental Sciences*, vol. 11, no. 23, 2025.
- [4] V. S. a. M. R. H. Ritchie, “Urbanization,” *Our World in Data*, 2018.
- [5] O. W. i. D. team, “Measuring progress towards the Sustainable Development Goals,” *Our World in Data*, 2023.
- [6] G. S. P. E. S. E. A. P. R. B. Jose Sanchez Gracias, “Smart Cities—A Structured Literature Review,” *MPDI*, vol. 6, no. 4, p. 1719–1743, 2023.
- [7] O. w. i. d. team, “Make cities inclusive, safe, resilient and sustainable,” *Our world in data*, 2023.
- [8] A. A. M. J. A. F. K. K. B. M. S. Fatima Tariq, “Security and Privacy Preservation in Smart Cities During Security Surveillance: A Mapping Study,” *Wiley*, no. 1, p. e6727022, 2025.
- [9] N. I. C. D. A. A. Almando Morain, “Artificial Intelligence for Water Consumption Assessment:,” *Water Resources Management*, vol. 38, p. 3113 – 3134, 2024.
- [10] K. S. J. T. X. X. Z. Yuxing Xie, “EXPLORING CROSS-CITY SEMANTIC SEGMENTATION OF ALS POINT CLOUDS,” in *XXIV ISPRS Congress*, 2021.
- [11] Q. C. Pinliang Dong, in *LiDAR Remote Sensing and Applications*, Boca Raton, Taylor & Francis, 2017, p. 1.
- [12] X. Z. S. B. S. L. P. Y. L. Vishal Thengane, “Foundational Models for 3D Point Clouds: A Survey and Outlook,” *MPDI*, 2025.
- [13] V. K. a. T. F. A. Golovinskiy, “Shape-based recognition of 3D point clouds in urban environments,” in *IEEE International Conference on Computer Vision (ICCV)*, 2009.

- [14] Z. X. J. W. T. L. L. W. G. S. L. T. Hesong Dong, "High-precision airborne LiDAR remains essential for urban forestry: Revealing the limitations of recent large-scale canopy height products in," *Elsevier*, vol. 143, pp. 6-8, 2025.
- [15] Y. B. a. A. C. Ian Goodfellow, "Deep learning," MIT press, 2017, pp. 96-97.
- [16] "Deep Learning for 3D Point Cloud Enhancement: A Survey," *Siwen Quan, Junhao Yu, Ziming Nie, Muze Wang, Sijia Feng, Pei An, and Jiaqi Yang*, pp. 1-3, 2024.
- [17] H. S. K. M. L. J. G. Charles R. Qi, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [18] M. G.-F. Andrey Dimitrov, "Segmentation of building point cloud models including detailed architectural/structural features and MEP systems," *Elsevier*, vol. 51, pp. 32-45, 2015.
- [19] R. B. M. S. W. W. X. D. B. C. Yangyan Li, "PointCNN: Convolution On X -Transformed Points," *NeurIPS*, 2018.
- [20] S. Y. C. W. Saifullahi Aminu Bello, "Review: deep learning on 3D point clouds," pp. 1-4, 2020.
- [21] M. C. F. S. M. B. J. L. Cheng Wang, "NormalNet: A voxel-based CNN for 3D object classification and," *Elsevier*, vol. 323, pp. 139-141, 2019.
- [22] L. Y. H. S. L. J. G. Charles R. Qi, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [23] C. R. Q. J.-E. D. B. M. F. G. L. J. G. Hugues Thomas, "KPConv: Flexible and Deformable Convolution for Point Clouds," in *International Conference on Computer Vision*, Seoul, 2019.
- [24] Y.-H. H. T. T. D. B. J. Z. Hugues Thomas, "KPConvX: Modernizing Kernel Point Convolution with Kernel Attention," in *Conference on Computer Vision and Pattern Recognition*, Toronto, 2024.
- [25] X. Z. S. L. J. D. Hang Gao, "Deformable Kernels: Adapting Effective Receptive Fields for Object Deformation," in *International Conference on Learning Representations (ICLR) 2020*, 2019.
- [26] Y. C. G. L. Y. S. X. Z. D. C. a. Y. Z. Jiakang Xia, "PGFormer: A Point Cloud Segmentation Network for Urban Scenes Combining Grouped Transformer and KPConv," *IEEE Transactions on Geoscience and Remote Sensing (TGRS)*, vol. 63, p. 5704318, 2025.

- [27] A. R. T. M. B. B. Maxim Berman, “The Lovasz-Softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks,” in *Conference on Computer Vision and Pattern Recognition*, Leuven, 2018.